

Syntax Parsing

Md Shad Akhtar

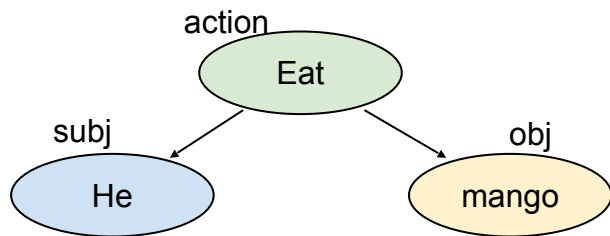
shad.akhtar@iiitd.ac.in



INDRAPRASTHA INSTITUTE *of*
INFORMATION TECHNOLOGY **DELHI**

Syntax Processing - Why do we need it?

- Consider a sentence
 - He eat mango. (*)
- Can we infer high-level semantic from it?
 - Most probably, YES.
- Why and how?
 - Higher-level cognitive semantic was invoked.
- It also tells us that some information are missing
 - tense [past, present, or future] and
 - number [one mango or many mangoes]



- Consider another sentence
 - He a mango eats. (*)
- Similarly, we can infer high-level semantic from it.
- We also get the number and tense information.

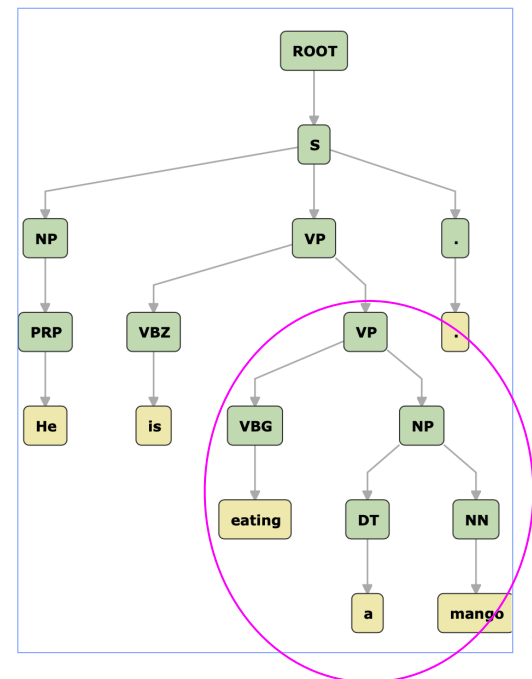
Whenever there is a mistake at the lower level, more processing is required at the higher level !!

Pragmatics & Discourse	<i>Study of semantics in context.</i>
Semantics	<i>Meaning of the sentence.</i>
Parsing	<i>Syntactic structure of the sentence.</i>
Chunking	<i>Grouping of meaningful phrases.</i>
Part of speech tagging	<i>Grammatical classes.</i>
Morphology	<i>Study of word structure.</i>



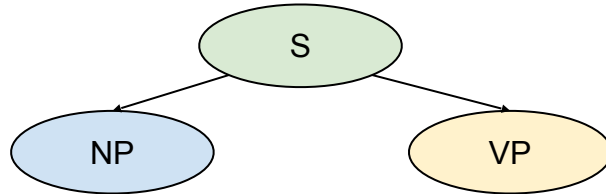
Syntax Processing

- A sentence has a sequential structure (sequence of words), then why hierarchical structure is required?
 - He is eating a mango.
- It helps in establishing relationship among constituents (words or phrases) of a sentence.
 - ‘*Eating*’ is the head of the verb phrase ‘*eating a mango*’, and it should take an object (noun phrase ‘*a mango*’ in this case).

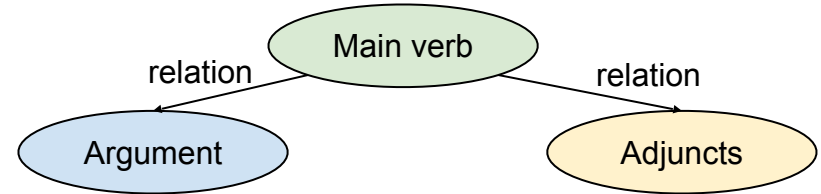


Two representations of parsing

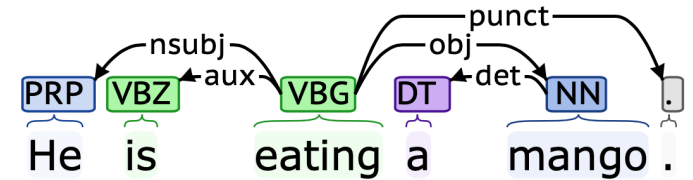
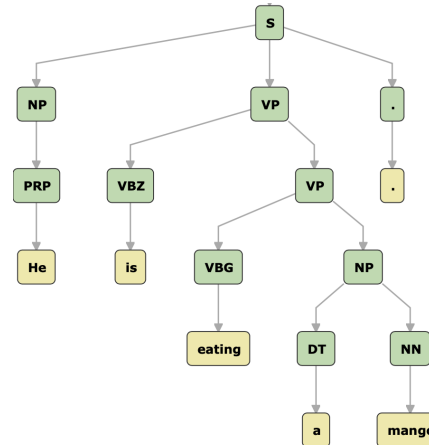
Constituency



Dependency

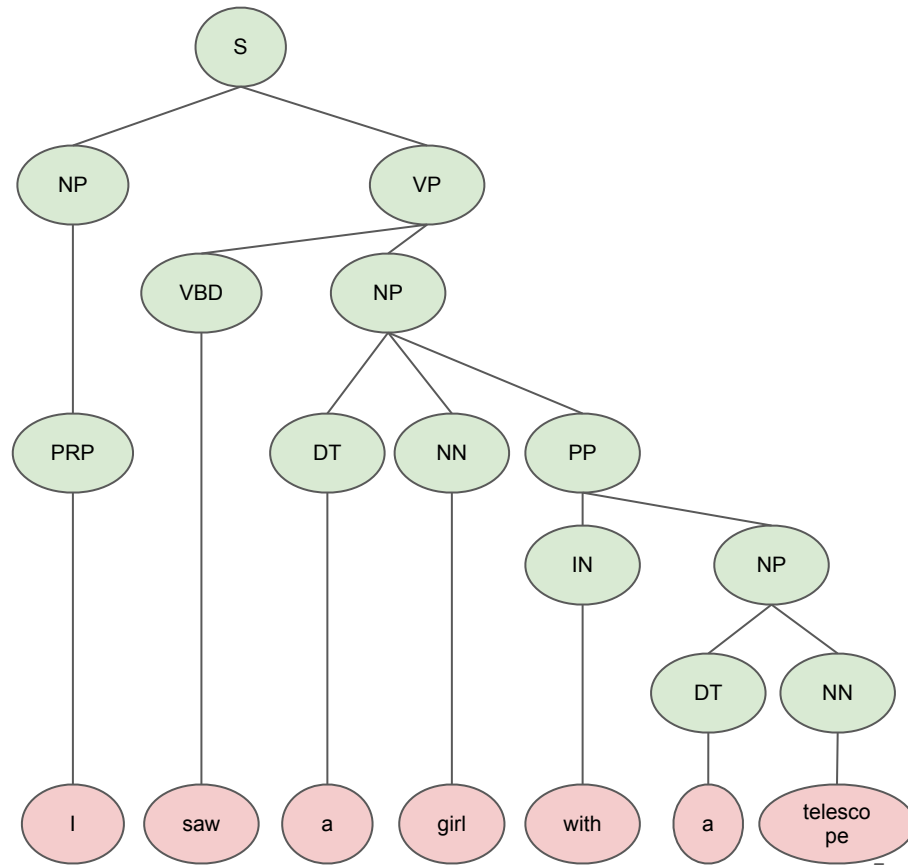
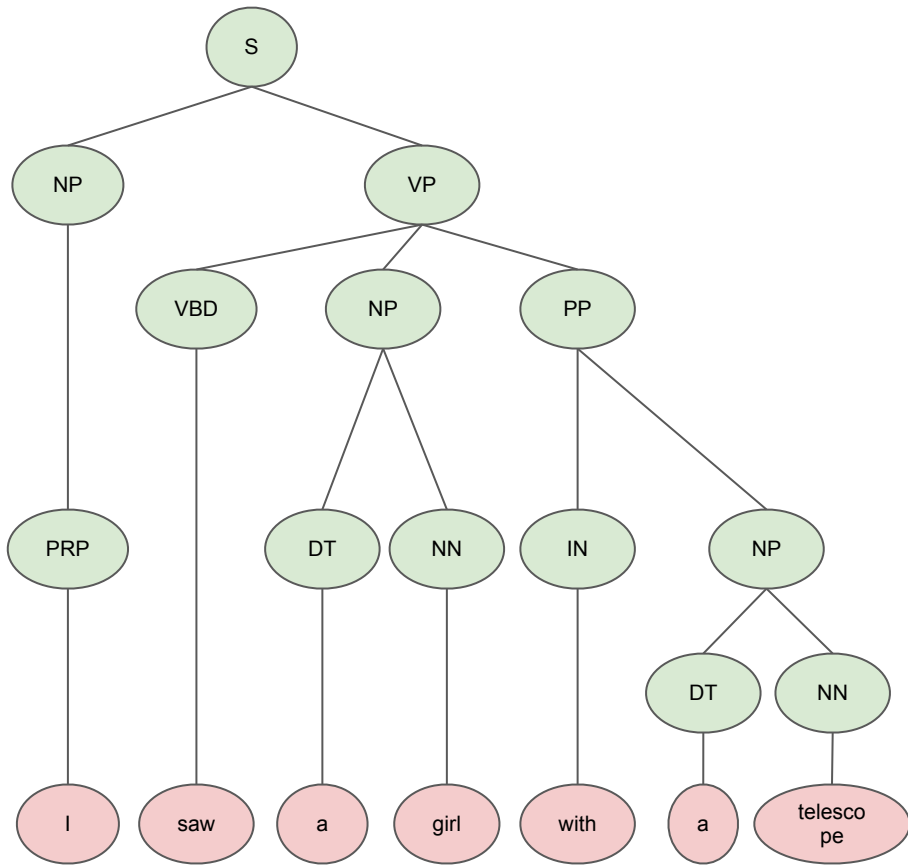


(S
(NP (PRP He))
(VP
(VBZ is)
(VP
(VBZ eating)
(NP
(DT a)
(NN mango)
)))))



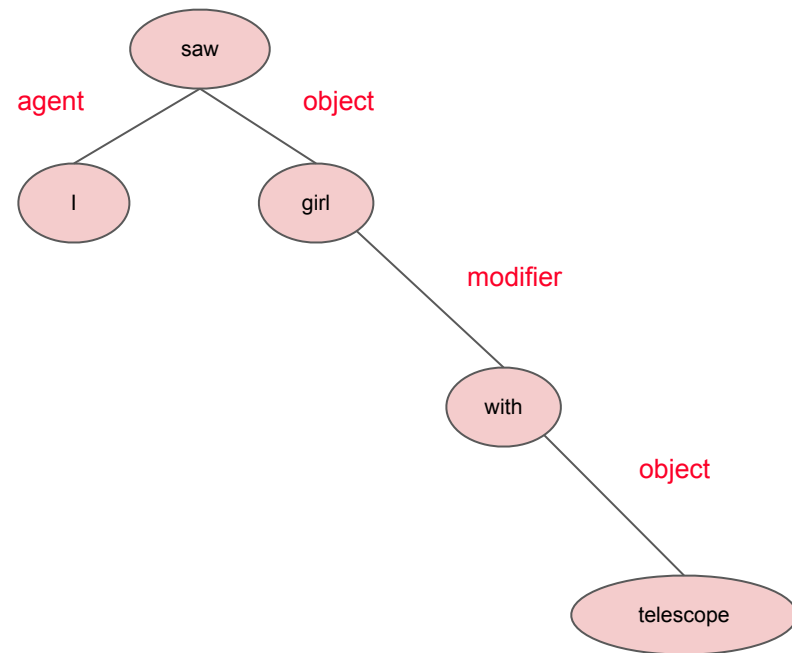
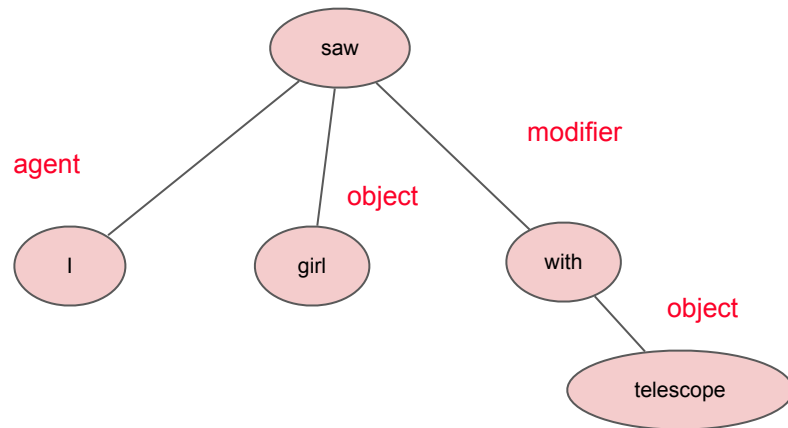


Constituency Parsing





Dependency Parsing



CFG: Some preliminaries

Context-free Grammar (CFG)

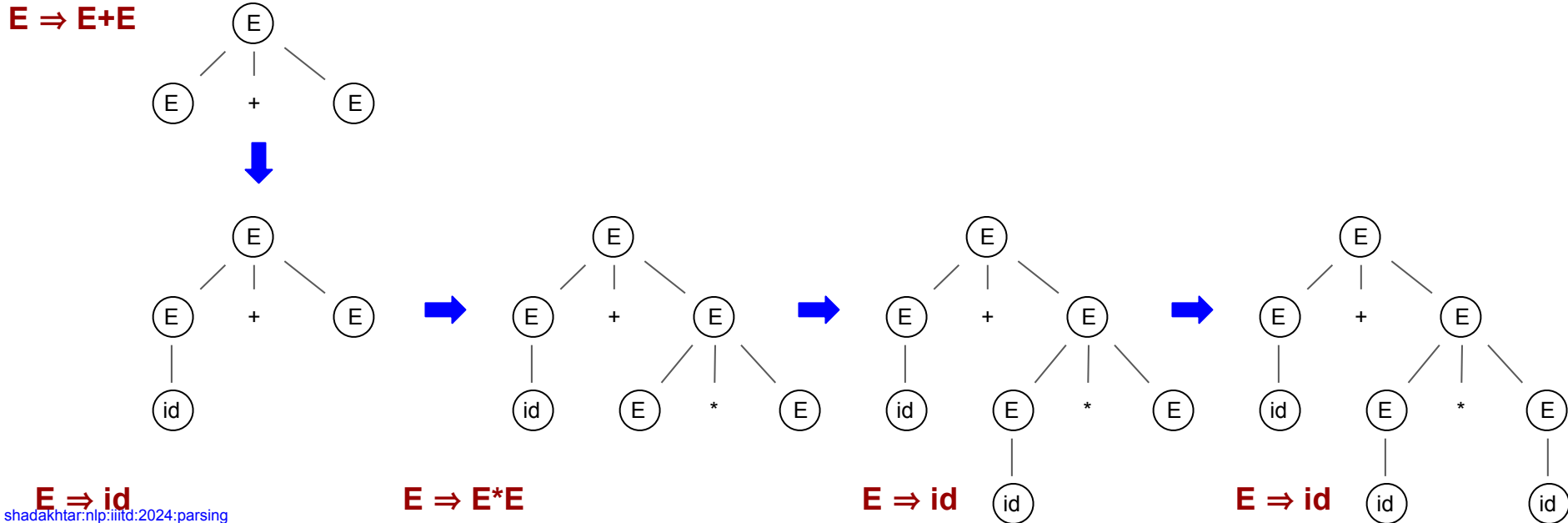
- Provides a precise syntactic specification of a language
- A CFG $G = \langle N, T, P, S \rangle$
 - **N**on-terminals:
 - A finite set of non-terminals (variables) [usually in capital letters]
 - **T**erminals:
 - A finite set of terminals (input symbols|tokens) [usually in small letters]
 - **P**roduction:
 - A finite set of productions rules in the following form $A \rightarrow \alpha$ where A is a non-terminal and α is a string of terminals and non-terminals (including the empty string); $|A| \leq |\alpha|$
 - **S**tart symbol:
 - One of the non-terminal symbols

CFG: An example

- CFG $G = \langle N, T, P, S \rangle$
 - Non-terminal = $\{E\}$
 - Terminals = $\{+, -, *, |, (,), \text{id}\}$
 - Start symbol = $\{E\}$
 - Production
 - $E \rightarrow E + E$
 - $E \rightarrow E - E$
 - $E \rightarrow E * E$
 - $E \rightarrow E | E$
 - $E \rightarrow - E$
 - $E \rightarrow (E)$
 - $E \rightarrow \text{id}$

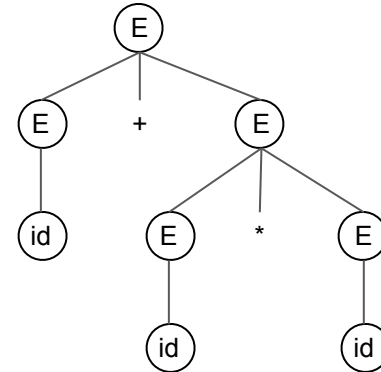
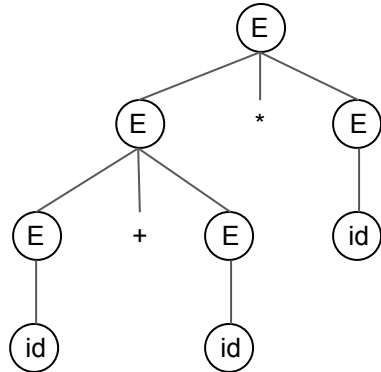
Parse Tree

- A graphical representation of a derivation
- Intermediate nodes: Inner nodes of a parse tree
- Leaves: Terminal symbols



Ambiguity

- A grammar that produces more than one parse tree for a sentence is called an ambiguous grammar
- $E \Rightarrow E+E \Rightarrow id+E \Rightarrow id+E*E \Rightarrow^* id+id*E \Rightarrow id+id*id$
- $E \Rightarrow E*E \Rightarrow E+E*E \Rightarrow id+E*E \Rightarrow id+id*E \Rightarrow id+id*id$



Ambiguity and Parser

- For most parsers, the grammar must be unambiguous.
 - unique selection of the parse tree for a sentence
- Disambiguation of an ambiguous grammar
 - Necessary to eliminate the ambiguity in the grammar
 - Choose one of the parse trees of a sentence to restrict to this choice

Left Recursion

- A grammar is left recursive if it has a non-terminal A such that there is a derivation
 - $A \Rightarrow^+ A\alpha$ for some string α
- Top-down parsing techniques **cannot handle** left-recursive grammars
- Possible ways of left-recursion
 - It may appear in a single step of the derivation (immediate left-recursion)
 - It may appear in more than one step of the derivation

Why left-recursion is a problem?

- Given

$$\blacksquare A \rightarrow Aa \mid b$$

generate a top-down parse tree from input string 'aaaaaa'

- On first input symbol 'a', you apply first production since second production expect first character to be 'b'. [Note that you don't know what's your second input]
 - $A \Rightarrow Aa \Rightarrow Aaa \Rightarrow \dots \Rightarrow Aaaaaaa$
 - We are waiting to reduce 'A' to 'a'
 - After infinite/many steps, we may get to know that the path we chose was not correct.

Removing Left Recursion

In general,

$A \rightarrow A\alpha_1 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \dots \mid \beta_n$ Where $\beta_1 \dots \beta_n$ do not start with A

$\Downarrow$

eliminate immediate left recursion

$A \rightarrow \beta_1 A' \mid \dots \mid \beta_n A'$

$A' \rightarrow \alpha_1 A' \mid \dots \mid \alpha_m A' \mid \varepsilon$

an equivalent grammar

Removing Left Recursion: An example

$$E \rightarrow E+T \mid T$$
$$T \rightarrow T*F \mid F$$
$$F \rightarrow \text{id} \mid (E)$$
$$\Downarrow$$

eliminate immediate left recursion

$$E \rightarrow T E'$$
$$E' \rightarrow +T E' \mid \varepsilon$$
$$T \rightarrow F T'$$
$$T' \rightarrow *F T' \mid \varepsilon$$
$$F \rightarrow \text{id} \mid (E)$$

Non-immediate Left-recursion

$$S \rightarrow Aa \mid b$$
$$A \rightarrow Sc \mid d$$

Left-recursion

$$S \Rightarrow Aa \Rightarrow Sca$$

Or

$$A \Rightarrow Sc \Rightarrow Aac$$

Parsing in English language

Top-Down Parsing

- Parse tree are created top to bottom
 - Begin with the start symbol to generate the input string.
- Top-down parser
 - Recursive-Descent Parsing
 - Predictive Parsing
 - Non-recursive Predictive Parsing (LL(1) parsing)

Recursive-Descent Parsing

- Recursively applies production rules
- If current production fails, *backtrack*, apply another rule.
- Its simple but not widely used
- Not efficient
 - Cost of backtracking is involved which may be huge.

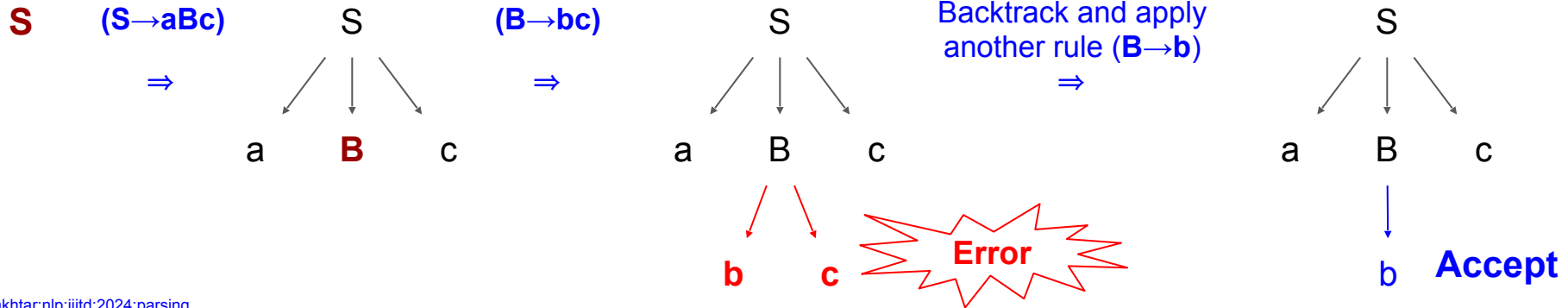
G: $S \rightarrow aBc$

$S \rightarrow AX$

$X \rightarrow BC$

$A \rightarrow a$

$C \rightarrow c$



A simple grammar

Productions (P) or rules:

S	→	NP VP
NP	→	PRP
NP	→	NN
NP	→	DT NN
NP	→	DT NN PP
VP	→	VBG NP
VP	→	VBZ VP
VP	→	VBD NP
VP	→	VBD NP PP
PP	→	IN NP
PP	→	DT NN PP

Lexicon

PRP	→
PRP	→
VBZ	→
VBD	→
VBG	→
DT	→
NN	→
NN	→
NN	→
IN	→

He
I
is
saw
eating
a
mango
girl
telescope
with

Can cover all three sentences and more..



Top-down parsing

1 He
2 is
3 eating
4 a
5 mango
6

Productions (P) or rules:

S	→	NP VP
NP	→	PRP
NP	→	NN
NP	→	DT NN
VP	→	VBG NP
VP	→	VBZ VP

Lexicon

PRP	→	He
VBZ	→	is
VBG	→	eating
DT	→	a
NN	→	mango

	State	Backup state	Action
1	[(S), #1]		
2	[(NP VP), #1]		
3	[(NN VP), #1]	[(PRP VP), #1] [(DT NP VP), #1]	NN doesn't match with 'He', so <i>backtrack</i>
3	[(PRP VP), #1]	[(DT NP VP), #1]	Consume 'He'
4	[(VP), #2]		
5	[(VBZ VP), #2]	[(VBG NP), #2]	Consume 'is'
6	[(VP), #3]		
7	[(VBZ VP), #3]	[(VBG NP), #3]	VBZ doesn't match with 'eating', so <i>backtrack</i>
7	[(VBG NP), #3]		Consume 'eating'
8	[(NP), #4]		
9	[(DT NN), #4]]	[(NN), #4]] [(PRP), #4]]	Consume 'a'. Similarly, consume 'mango'
10	[(), #6]		Input consumed and no symbol remaining. Accept

Bottom-Up Parsing

- Construct a parse tree for an input string beginning at the leaves (the bottom) and working up towards the root (the top)
 - Reducing a string w to the start symbol of a grammar.
 - At each reduction step, a particular substring matching the right side of a production is replaced by the symbol on the left of that production.
 - Gives the right-most derivation in the reverse order.

Top-down vs Bottom-up Parsing

S →	NP VP
NP →	PRP
NP →	NN
NP →	DT NN
VP →	VBG NP
VP →	VBZ VP
PRP →	He
VBZ →	is
VBG →	eating
DT →	a
NN →	mango

- Top-down
 - $S \Rightarrow NP VP \Rightarrow PRP VP \Rightarrow He VP \Rightarrow He VBZ VP \Rightarrow He is VP \Rightarrow He is VBG NP \Rightarrow He is eating NP \Rightarrow He is eating DT NN \Rightarrow He is eating a NN \Rightarrow He is eating a mango$
- Bottom-up
 - $He is eating a mango \Rightarrow PRP is eating a mango \Rightarrow NP is eating a mango \Rightarrow NP VBZ eating a mango \Rightarrow NP VBZ VBG a mango \Rightarrow NP VBZ VBG DT mango \Rightarrow NP VBZ VBG DT NN \Rightarrow NP VBZ VBG NP \Rightarrow NP VBZ VP \Rightarrow NP VP \Rightarrow S$

Bottom-up approach

- Notations

- Star (*) defines the position of processing.

[Note: Convention is to use dot (.). For illustration, we'll use *]

S → NP * VP

NP has been processed and VP is waiting to be processed

1 He
2 is
3 eating
4 a
5 mango
6

Bottom-up parsing

Productions (P) or rules:		
S	→	NP VP
NP	→	PRP
NP	→	NN
NP	→	DT NN
VP	→	VBG NP
VP	→	VBZ VP
Lexicon		
PRP	→	He
VBZ	→	is
VBG	→	eating
DT	→	a
NN	→	mango

Shift [* He is eating a mango]

Reduce [He * is eating a mango]

Reduce [PRP * is eating a mango]

Reduce [NP * is eating a mango]

Shift [NP is * eating a mango]

Reduce [NP VBZ * eating a mango]

Shift [NP VBZ eating * a mango]

Reduce [NP VBZ VBG * a mango]

Shift [NP VBZ VBG a * mango]

Reduce [NP VBZ VBG DT * mango]

Shift [NP VBZ VBG DT mango *]

Reduce [NP VBZ VBG DT NN *]

Reduce [NP VBZ VBG NP *]

Reduce [NP VBZ VP *]

Reduce [NP VP *]

Reduce [S *]

Start symbol reached, no input symbols remaining. Accept

Properties of Top-down and Bottom-up parsing

- Top-down
 - No regards to data
 - Can't handle left-recursive grammar
 - Predicts (At each step, it predicts that the expanded production will work.)
 - Need to backtrack
- Bottom-up
 - Data driven
 - If a word can take multiple tags (which usually do), then we may have to expand unnecessary productions.

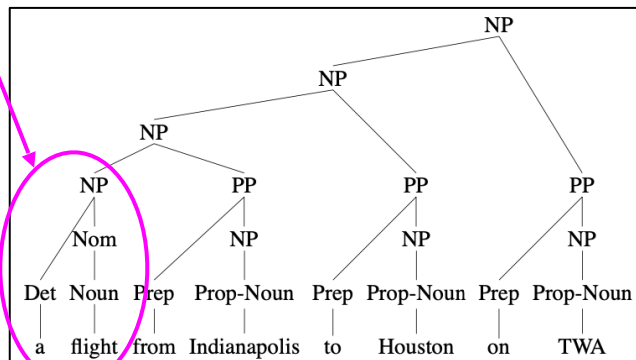
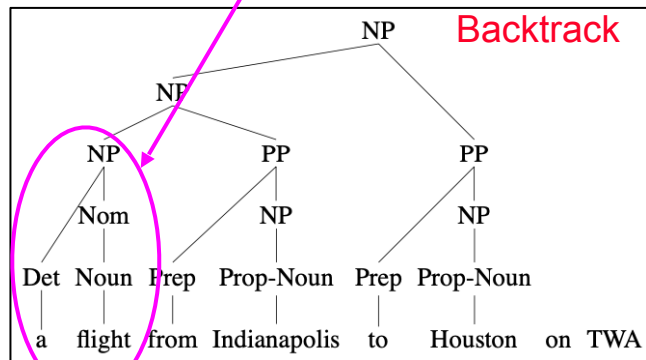
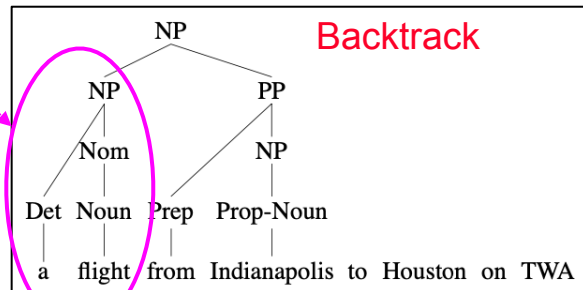
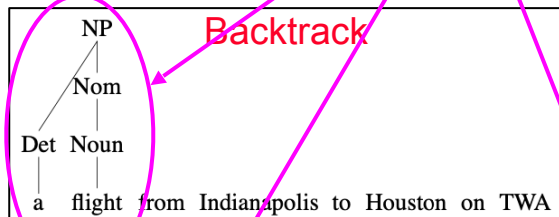
Other parsing algorithms

- CYK [Cocke, Younger, Kashmi] parser
 - Dynamic Programming
 - Bottom-up
 - Requires grammar in Chomsky Normal Form (CNF)
 - Production rules can have two forms only
 - $A \rightarrow BC$ (Two non-terminals on the right side of production)
 - $A \rightarrow a$ (Single terminal on the right side of production)
- Earley's parser
 - Dynamic Programming
 - Top-down
 - Can handle left-recursive and (not efficiently) ambiguous grammar

Why Dynamic Programming?

With DP, we can compute it once and save it for reuse.

Have to perform the same operation 4 times!



CYK Parser

- S → NP VP
- NP → NN | DT NN | NP PP
- VP → VBD NP
- VP → VP PP
- PP → IN NP
- NN → building | bullets |
- gunman → sprayed
- VBD → the
- DT → with
- IN

1 The 2 gunman 3 sprayed 4 the 5 building 6 with 7 bullets 8

		To						
		2	3	4	5	6	7	8
From	1							
	2							
	3							
	4							
	5							
	6							
	7							

CYK parser

- S → NP VP
- NP → NN | DT NN | NP PP
- VP → VBD NP
- VP → VP PP
- PP → IN NP
- NN → building | bullets |
- gunman
- VBD → sprayed
- DT → the
- IN → with

1 The 2 gunman 3 sprayed 4 the 5 building 6 with 7 bullets 8

→ sprayed → the → with		To						
		2	3	4	5	6	7	8
From	1	DT ₁₂						
	2							
	3							
	4							
	5							
	6							
	7							

- S → NP VP
- NP → NN | DT NN | NP PP
- VP → VBD NP
- VP → VP PP
- PP → IN NP
- NN → building | bullets | gunman
- VBD → sprayed
- DT → the
- IN → with

1 The 2 gunman 3 sprayed 4 the 5 building 6 with 7 bullets 8

		To						
		2	3	4	5	6	7	8
From	1	DT ₁₂						
	2		NN ₂₃					
	3							
	4							
	5							
	6							
	7							

- S → NP VP
- NP → NN | DT NN | NP PP
- VP → VBD NP
- VP → VP PP
- PP → IN NP
- NN → building | bullets |
- gunman
- VBD → sprayed
- DT → the
- IN → with

1 The 2 gunman 3 sprayed 4 the 5 building 6 with 7 bullets 8

		To						
		2	3	4	5	6	7	8
From	1	DT ₁₂	NP ₁₃					
	2		NN ₂₃					
	3							
	4							
	5							
	6							
	7							

CYK parser

- S → NP VP
- NP → NN | DT NN | NP PP
- VP → VBD NP
- VP → VP PP
- PP → IN NP
- NN → building | bullets |
- gunman
- VBD → sprayed
- DT → the
- IN → with

1 The 2 gunman 3 sprayed 4 the 5 building 6 with 7 bullets 8

		To						
		2	3	4	5	6	7	8
From	1	DT ₁₂	NP ₁₃					
	2		NN ₂₃					
	3		VBD ₃₄					
	4							
	5							
	6							
	7							

CYK parser

- S → NP VP
- NP → NN | DT NN | NP PP
- VP → VBD NP
- VP → VP PP
- PP → IN NP
- NN → building | bullets |
- gunman → sprayed
- VBD → the
- DT → with
- IN

1 The 2 gunman 3 sprayed 4 the 5 building 6 with 7 bullets 8

		To						
		2	3	4	5	6	7	8
From	1	DT ₁₂	NP ₁₃					
	2		NN ₂₃	--				
	3			VBD ₃₄				
	4							
	5							
	6							
	7							

CYK parser

- S → NP VP
- NP → NN | DT NN | NP PP
- VP → VBD NP
- VP → VP PP
- PP → IN NP
- NN → building | bullets | gunman
- VBD → sprayed
- DT → the
- IN → with

1 The 2 gunman 3 sprayed 4 the 5 building 6 with 7 bullets 8

→ sprayed → the → with		To						
		2	3	4	5	6	7	8
From	1	DT ₁₂	NP ₁₃	--				
	2		NN ₂₃	--				
	3			VBD ₃₄				
	4							
	5							
	6							
	7							

- S → NP VP
- NP → NN | DT NN | NP PP
- VP → VBD NP
- VP → VP PP
- PP → IN NP
- NN → building | bullets |
- gunman → sprayed
- VBD → the
- DT → with
- IN

1 The 2 gunman 3 sprayed 4 the 5 building 6 with 7 bullets 8

→ sprayed → the → with		To						
		2	3	4	5	6	7	8
From	1	DT ₁₂	NP ₁₃	--				
	2		NN ₂₃	--				
	3			VBD ₃₄				
	4				DT ₄₅			
	5							
	6							
	7							

CYK parser

- S → NP VP
- NP → NN | DT NN | NP PP
- VP → VBD NP
- VP → VP PP
- PP → IN NP
- NN → building | bullets |
- gunman → sprayed
- VBD → the
- DT → with
- IN

1 The 2 gunman 3 sprayed 4 the 5 building 6 with 7 bullets 8

		To						
		2	3	4	5	6	7	8
From	1	DT ₁₂	NP ₁₃	--	--			
	2		NN ₂₃	--	--			
	3			VBD ₃₄	--			
	4				DT ₄₅			
	5					NN ₅₆		
	6							
	7							

CYK parser

- S → NP VP
- NP → NN | DT NN | NP PP
- VP → VBD NP
- VP → VP PP
- PP → IN NP
- NN → building | bullets |
- gunman → sprayed
- VBD → the
- DT → with
- IN

1 The 2 gunman 3 sprayed 4 the 5 building 6 with 7 bullets 8

		To						
		2	3	4	5	6	7	8
From	1	DT ₁₂	NP ₁₃	--	--			
	2		NN ₂₃	--	--			
	3			VBD ₃₄	--			
	4				DT ₄₅	NP ₄₆		
	5					NN ₅₆		
	6							
	7							

CYK parser

- S → NP VP
- NP → NN | DT NN | NP PP
- VP → VBD NP
- VP → VP PP
- PP → IN NP
- NN → building | bullets |
- gunman → sprayed
- VBD → the
- DT → with
- IN

1 The 2 gunman 3 sprayed 4 the 5 building 6 with 7 bullets 8

		To						
		2	3	4	5	6	7	8
From	1	DT ₁₂	NP ₁₃	--	--			
	2		NN ₂₃	--	--			
	3			VBD ₃₄	--	VP ₃₆		
	4				DT ₄₅	NP ₄₆		
	5					NN ₅₆		
	6							
	7							

CYK parser

- S → NP VP
- NP → NN | DT NN | NP PP
- VP → VBD NP
- VP → VP PP
- PP → IN NP
- NN → building | bullets |
- gunman
- VBD → sprayed
- DT → the
- IN → with

1 The 2 gunman 3 sprayed 4 the 5 building 6 with 7 bullets 8

		To						
		2	3	4	5	6	7	8
From	1	DT ₁₂	NP ₁₃	--	--	S ₁₆		
	2		NN ₂₃	--	--	--		
	3			VBD ₃₄	--	VP ₃₆		
	4				DT ₄₅	NP ₄₆		
	5					NN ₅₆		
	6							
	7							

CYK parser

- S → NP VP
- NP → NN | DT NN | NP PP
- VP → VBD NP
- VP → VP PP
- PP → IN NP
- NN → building | bullets | gunman
- VBD → sprayed
- DT → the
- IN → with

1 The 2 gunman 3 sprayed 4 the 5 building 6 with 7 bullets 8

		To						
		2	3	4	5	6	7	8
From	1	DT ₁₂	NP ₁₃	--	--	S ₁₆		
	2			NN ₂₃	--	--	--	
	3			VBD ₃₄	--	VP ₃₆		
	4					DT ₄₅	NP ₄₆	
	5					NN ₅₆		
	6						IN ₆₇	
	7							

CYK parser

- S → NP VP
- NP → NN | DT NN | NP PP
- VP → VBD NP
- VP → VP PP
- PP → IN NP
- NN → building | bullets |
- gunman → sprayed
- VBD → the
- DT → with
- IN

1 The 2 gunman 3 sprayed 4 the 5 building 6 with 7 bullets 8

		To						
		2	3	4	5	6	7	8
From	1	DT ₁₂	NP ₁₃	--	--	S ₁₆	--	
	2		NN ₂₃	--	--	--	--	
	3			VBD ₃₄	--	VP ₃₆	--	
	4				DT ₄₅	NP ₄₆	--	
	5					NN ₅₆	--	
	6						IN ₆₇	
	7							NN ₇₈

CYK parser

- S → NP VP
- NP → NN | DT NN | NP PP
- VP → VBD NP
- VP → VP PP
- PP → IN NP
- NN → building | bullets |
- gunman → sprayed
- VBD → the
- DT → with
- IN

1 The 2 gunman 3 sprayed 4 the 5 building 6 with 7 bullets 8

		To						
		2	3	4	5	6	7	8
From	1	DT ₁₂	NP ₁₃	--	--	S ₁₆	--	
	2		NN ₂₃	--	--	--	--	
	3			VBD ₃₄	--	VP ₃₆	--	
	4				DT ₄₅	NP ₄₆	--	
	5					NN ₅₆	--	
	6						IN ₆₇	
	7							NP ₇₈ NN ₇₈

CYK parser

- S → NP VP
- NP → NN | DT NN | NP PP
- VP → VBD NP
- VP → VP PP
- PP → IN NP
- NN → building | bullets |
- gunman → sprayed
- VBD → the
- DT → with
- IN

1 The 2 gunman 3 sprayed 4 the 5 building 6 with 7 bullets 8

		To						
		2	3	4	5	6	7	8
From	1	DT ₁₂	NP ₁₃	--	--	S ₁₆	--	
	2		NN ₂₃	--	--	--	--	
	3			VBD ₃₄	--	VP ₃₆	--	
	4				DT ₄₅	NP ₄₆	--	
	5					NN ₅₆	--	
	6						IN ₆₇	PP ₆₈
	7							NP ₇₈ NN ₇₈

- S → NP VP
- NP → NN | DT NN | NP PP
- VP → VBD NP
- VP → VP PP
- PP → IN NP
- NN → building | bullets |
- gunman → sprayed
- VBD → the
- DT → with
- IN

1 The 2 gunman 3 sprayed 4 the 5 building 6 with 7 bullets 8

		To						
		2	3	4	5	6	7	8
From	1	DT ₁₂	NP ₁₃	--	--	S ₁₆	--	
	2		NN ₂₃	--	--	--	--	
	3			VBD ₃₄	--	VP ₃₆	--	
	4				DT ₄₅	NP ₄₆	--	NP ₄₈
	5					NN ₅₆	--	--
	6						IN ₆₇	PP ₆₈
	7							NP ₇₈ NN ₇₈

CYK parser

S	→ NP VP
NP	→ NN DT NN NP PP
VP	→ VBD NP
VP	→ VP PP
PP	→ IN NP
NN	→ building bullets
gunman	
VBD	→ sprayed
DT	→ the
IN	→ with

1 The 2 gunman 3 sprayed 4 the 5 building 6 with 7 bullets 8

		To						
		2	3	4	5	6	7	8
From	1	DT ₁₂	NP ₁₃	--	--	S ₁₆	--	
	2		NN ₂₃	--	--	--	--	
	3			VBD ₃₄	--	VP ₃₆	--	VP ₃₈
	4				DT ₄₅	NP ₄₆	--	NP ₄₈
	5					NN ₅₆	--	--
	6						IN ₆₇	PP ₆₈
	7							NP ₇₈ NN ₇₈

CYK parser

S	→ NP VP
NP	→ NN DT NN NP PP
VP	→ VBD NP
VP	→ VP PP
PP	→ IN NP
NN	→ building bullets
gunman	
VBD	→ sprayed
DT	→ the
IN	→ with

1 The 2 gunman 3 sprayed 4 the 5 building 6 with 7 bullets 8

		To						
		2	3	4	5	6	7	8
From	1	DT ₁₂	NP ₁₃	--	--	S ₁₆	--	
	2		NN ₂₃	--	--	--	--	
	3			VBD ₃₄	--	VP ₃₆	--	VP ₃₈ VP ₈₈
	4				DT ₄₅	NP ₄₆	--	NP ₄₈
	5					NN ₅₆	--	--
	6						IN ₆₇	PP ₆₈
	7							NP ₇₈ NN ₇₈

→ sprayed
→ the
→ with

CYK parser

CYK parser

- S → NP VP
- NP → NN | DT NN | NP PP
- VP → VBD NP
- VP → VP PP
- PP → IN NP
- NN → building | bullets |
- gunman → sprayed
- VBD → the
- DT → with
- IN

1 The 2 gunman 3 sprayed 4 the 5 building 6 with 7 bullets 8

→ sprayed

→ the

→ with

2

3

4

5

6

7

8

1

2

3

4

5

6

7

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

--

--

--

--

--

--

S₁₆

--

VP₃₆

S₁₈

--

VP₃₈

NP₄₈

--

PP₆₈

DT₁₂

NN₂₃

VBD₃₄

DT₄₅

NN₅₆

IN₆₇

NP₁₃

NP₄₆

NP₇₈
NN₇₈

CYK parser

		To						
		2	3	4	5	6	7	8
From	1	DT ₁₂	NP ₁₃	--	--	S ₁₆	--	S ₁₈
	2		NN ₂₃	--	--	--	--	--
	3			VBD ₃₄	--	VP ₃₆	--	VP ₃₈
	4				DT ₄₅	NP ₄₆	--	NP ₄₈
	5					NN ₅₆	--	--
	6						IN ₆₇	PP ₆₈
	7							NP ₇₈ NN ₇₈

		To						
		2	3	4	5	6	7	8
From	1	DT ₁₂	NP ₁₃	--	--	S ₁₆	--	S ₁₈
	2		NN ₂₃	--	--	--	--	--
	3			VBD ₃₄	--	VP ₃₆	--	VP ₃₈
	4				DT ₄₅	NP ₄₆	--	NP ₄₈
	5					NN ₅₆	--	--
	6						IN ₆₇	PP ₆₈
	7							NP ₇₈ NN ₇₈

Probabilistic Parsing

CFG Independence Assumption

- In context free grammars (CFG), any production (e.g., $NP \rightarrow NP PP$) can be applied irrespective of their context.
- However, it is not supported by data.
- PennTree bank
 - [Example from Dan Klein]

All NPs

$NP \rightarrow NP PP$	11%
$NP \rightarrow DT NN$	9%
$NP \rightarrow PRP$	6%

NPs under S

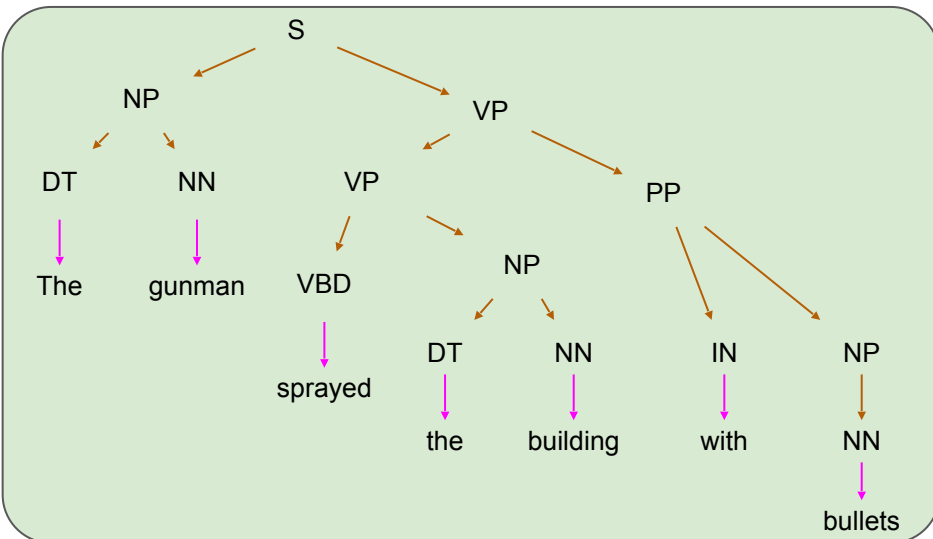
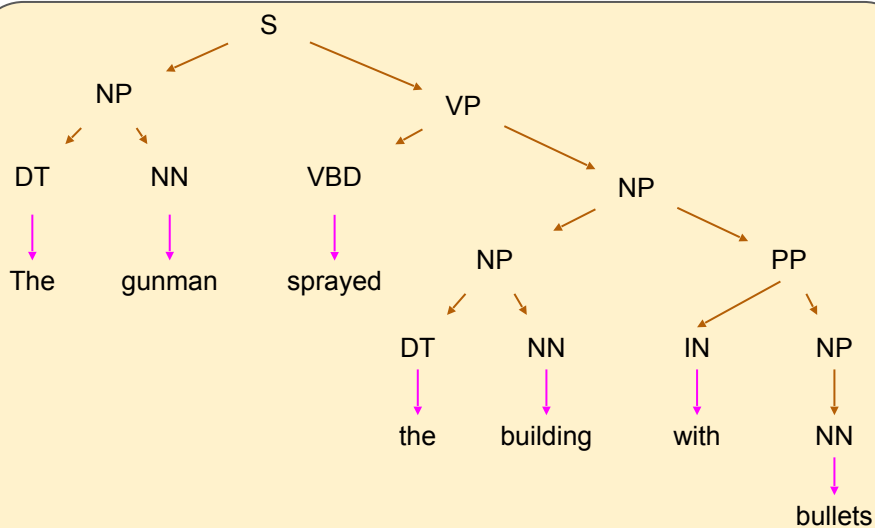
$NP \rightarrow NP PP$	9%
$NP \rightarrow DT NN$	9%
$NP \rightarrow PRP$	21%

NPs under VP

$NP \rightarrow NP PP$	23%
$NP \rightarrow DT NN$	7%
$NP \rightarrow PRP$	4%

Ambiguous CFG

- The gunman sprayed the building with bullets.
 - Two parse trees.
- Which one is more probable?
 - The parse tree which has the highest probability value



Probabilistic Context-free Grammar (PCFG)

- PCFG $G = \langle N, T, P, S \rangle$
 - N on-terminals:
 - T erminals:
 - S tart symbol:
 - P roduction:
 - A finite set of productions rules in the following form $A \rightarrow \alpha [p]$ where A is a non-terminal and α is a string of terminals and non-terminals (including the empty string); $|A| \leq |\alpha|$
 - p is the probability of the rule

Probabilistic Context-free Grammar (PCFG)

- S → NP VP [1.0]
- NP → NN [0.3]
- NP → DT NN [0.5]
- NP → NP PP [0.2]
- VP → VBD NP [0.4]
- VP → VP NP [0.6]
- PP → IN NP [1.0]
- NN → building [0.3]
- NN → bullets [0.3]
- NN → gunman [0.3]
- VBD → sprayed [1.0]
- DT → the [1.0]
- IN → with [1.0]

$$\forall_A \sum_j P(A \rightarrow B_j) = 1$$

Probability of a sentence and parse tree

- Probability of a sentence S

$$\begin{aligned} P(S) &= \sum_{T \in \{T'\}} P(S, T) \\ &= \sum_{T \in \{T'\}} P(T) \end{aligned}$$

where $T' = \{\text{All parse trees}\}$

- Probability of a parse tree T

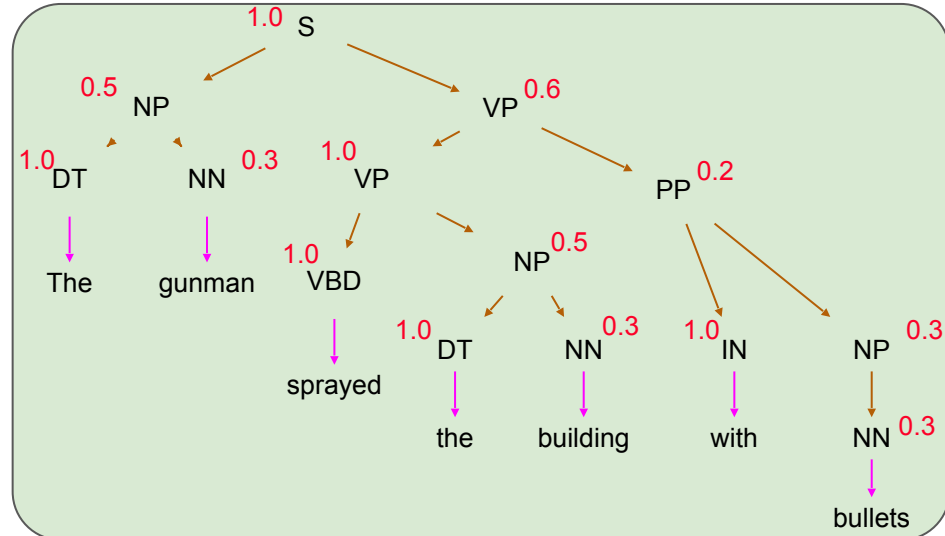
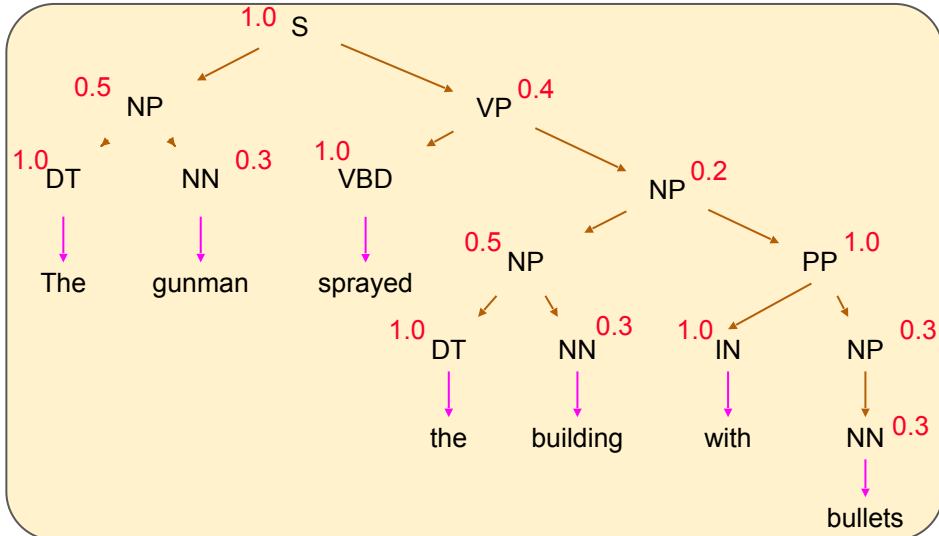
$$P(T) = \prod_i^n P_i(A \rightarrow B)$$

where n is the set of productions used in the tree.

Probability of the parse trees

$$\begin{aligned}
 P(T^1) &= P(S \rightarrow NP VP) \cdot P(NP \rightarrow DT NN) \cdot P(DT \rightarrow The) \cdot P(NN \rightarrow gunman) \cdot P(VP \rightarrow VBD NP) \cdot P(VBD \rightarrow sprayed) \cdot P(NP \rightarrow NP PP) \\
 &\quad P(NP \rightarrow DT NN) \cdot P(DT \rightarrow the) \cdot P(NN \rightarrow building) \cdot P(PP \rightarrow IN NP) \cdot P(IN \rightarrow with) \cdot P(NP \rightarrow NN) \cdot P(NN \rightarrow bullets) \\
 &= 1.0 * 0.5 * 0.3 * 0.4 * 0.2 * 0.5 * 0.3 * 0.3 * 0.3 = 0.000162
 \end{aligned}$$

$$\begin{aligned}
 P(T^2) &= P(S \rightarrow NP VP) \cdot P(NP \rightarrow DT NN) \cdot P(DT \rightarrow The) \cdot P(NN \rightarrow gunman) \cdot P(VP \rightarrow VP PP) \cdot P(VP \rightarrow VBD NP) \cdot P(VBD \rightarrow sprayed) \\
 &\quad P(NP \rightarrow DT NN) \cdot P(DT \rightarrow the) \cdot P(NN \rightarrow building) \cdot P(PP \rightarrow IN NP) \cdot P(IN \rightarrow with) \cdot P(NP \rightarrow NN) \cdot P(NN \rightarrow bullets) \\
 &= 1.0 * 0.5 * 0.3 * 0.6 * 0.5 * 0.3 * 0.2 * 0.3 * 0.3 = 0.000243
 \end{aligned}$$



Probabilistic parsing algorithms

- Using probabilistic CYK algorithm find out which one is the most probable interpretation for the given sentence.

S	→ NP VP	[1.0]
NP	→ PRP	[0.3]
NP	→ DT NN	[0.5]
NP	→ NP PP	[0.2]
VP	→ VBD NP	[0.4]
VP	→ VP NP	[0.6]
PP	→ IN NP	[1.0]
PRP	→ I	[1.0]
NN	→ telescope	[0.5]
NN	→ girl	[0.5]
VBD	→ saw	[1.0]
DT	→ a	[1.0]
IN	→ with	[1.0]

I saw a girl with a telescope.



OR

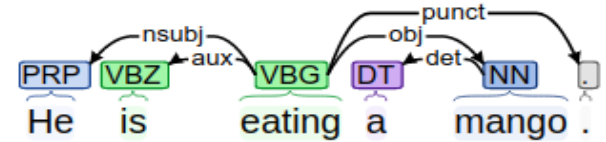


Add appropriate productions and associated probability values, if required.

Dependency Parsing

Dependency Structure

- Relation between words
 - rel_name(word1, word2)
 - det(mango, a)
- Depending upon the relation, one of the words acts as the head (governor) and other as the dependent (modifier)
- Head
 - Main verb in verb phrase or primary noun in the noun phrase, etc.
- Dependent
 - Remaining words that have direct or indirect connection with the head word in the phrase.



Dependency relations: A subset

Clausal Argument Relations	Description
NSUBJ	Nominal subject
DOBJ	Direct object
IOBJ	Indirect object
CCOMP	Clausal complement
XCOMP	Open clausal complement
Nominal Modifier Relations	Description
NMOD	Nominal modifier
AMOD	Adjectival modifier
NUMMOD	Numeric modifier
APPOS	Appositional modifier
DET	Determiner
CASE	Prepositions, postpositions and other case markers
Other Notable Relations	Description
CONJ	Conjunct
CC	Coordinating conjunction

Universal Dependency Set [(de Marneffe et al., 2014)]

MaltParser [Nivre et al., 2003, 2007]

- Discriminative model
- Based on Shift-Reduce parser
- Introduces two new operators
 - LeftArc and RightArc
- Maintains a stack and a buffer
- Asserts relation for the word at the top of the stack.

MaltParser [Nivre et al., 2003, 2007]

- Operation

- **LeftArc_X**: Assigns relation (**X**) between the word (head) at the top of the stack and the word (dependent) directly beneath it. Removes the word (dependent) after processing.

Stack[Root, w_{i-1} , w_i] Buffer[w_j , ...] **LeftArc_X** $\Rightarrow$ **X**($w_{i-1} \leftarrow w_i$)

Stack[Root, w_i] Buffer[w_j , ...]

- **RightArc_X**: Assigns relation (**X**) between the second word at the top of the stack and the first word (dependent) at the top of the buffer. Removes the top of the stack word (dependent) after processing.

Stack[Root, w_{i-1} , w_i] Buffer[w_j , ...] **RightArc_X** $\Rightarrow$ **X**($w_{i-1} \rightarrow w_i$)

Stack[Root, w_{i-1}] Buffer[w_j , ...]

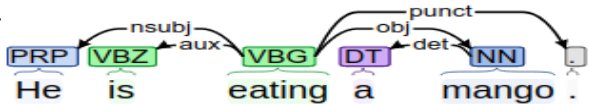
- **Shift**: Transfers the front word from buffer to top of the stack

Stack[Root, w_i] Buffer[w_j , w_{j+1} , ...] **Shift** $\Rightarrow$ Stack[Root, w_i , w_j] Buffer[w_{j+1} , ...]

An example

	Stack	Buffer	Action	Relation
0	[root]	[He, is, eating, a, mango, .]	Shift	
1	[root, He]	[is, eating, a, mango, .]	Shift	
2	[root, He, is]	[eating, a, mango, .]	Shift	
3	[root, He, is, eating]	[a, mango, .]	LeftArc _{Aux}	Aux(is ← eating)
4	[root, He, eating]	[a, mango, .]	LeftArc _{nSubj}	nSubj(He ← eating)
5	[root, eating, a]	[mango, .]	Shift	
6	[root, eating, a, mango]	[.]	LeftArc _{det}	det(a ← mango)
7	[root, eating, mango]	[.]	RightArc _{obj}	obj(eating → mango)
8	[root, eating, .]	[]	Shift	
9	[root, eating, .]	[]	RightArc _{punct}	punct(eating → .)
10	[root, eating]	[]	RightArc _{root}	root(root → eating)

Each action and relation are decided using
a discriminative model (e.g., SVM, etc.)



Thanks